



Competencia Olímpica – El Triatlón

El triatlón es un deporte que implica la realización de 3 disciplinas deportivas, natación, ciclismo y carrera a pie, que se realizan en orden y sin interrupción entre una prueba y la siguiente. El orden a realizar las pruebas es:



El tiempo que el triatleta tarda en cambiar de una disciplina a la siguiente se denomina transición; la transición de natación a ciclismo se conoce como T1 y la de ciclismo a carrera a pie como T2.



Las carreras de triatlón se clasifican en varias modalidades según:



- su distancia habiendo corta y larga distancia,
- si el circuito de ciclismo es en carretera o cross (por caminos y montaña) y
- si permiten el drafting: Cuando un ciclista avanza pedaleando por la carretera, la resistencia del aire ejerce presión sobre él, frenándolo. Al abrirse paso a través del aire, deja tras de sí una estela de aire turbulento, similar a la que deja un barco en el agua. Generalmente, la turbulencia



es mayor justo detrás del ciclista, y cuanto más lejos se encuentre la estela, menor será su turbulencia. Esta turbulencia en la corriente de aire ofrece menor resistencia que el aire en reposo a los ciclistas que entran en ella desde atrás, lo que significa que el ciclista que viene detrás necesita menos potencia para atravesarla a una velocidad determinada. Este es el concepto de drafting o cola de ciclista. Los pelotones de ciclismo profesional ruedan en grupo y comparten el esfuerzo de ir en cabeza, lo que les permite alcanzar velocidades medias mucho mayores que las de un ciclista solitario. Si bien en algunas modalidades del triatlón se permite el drafting, como en las pruebas de élite de la Serie Mundial de Triatlón y en los Juegos Olímpicos, en la mayoría de los triatlones no está permitido. Este deporte se concibe como un esfuerzo individual, sin colaboración para obtener una ventaja injusta sobre los demás. (Fuente: <https://www.raceranger.com/>)

Dentro de cada modalidad las competiciones más conocidas son:

- 1) Series mundiales de triatlón (conocidas como WTS por sus siglas en inglés): Campeonato mundial de varias carreras de corta distancia donde el ciclismo es en carretera y con drafting. El ganador del campeonato es el triatleta que obtenga más puntos tomando los 5 mejores resultados obtenidos en las pruebas de la temporada.
- 2) Ironman: Campeonato de distancia ironman donde no se permite el drafting. El ganador del campeonato es el vencedor del Ironman de Kona. Las plazas para correr este triatlón se otorgan a los primeros clasificados en alguna de las otras pruebas del circuito Ironman.
- 3) Xterra: Campeonato de corta distancia en modalidad triatlón cross. El ganador del campeonato es el vencedor del Triatlón de Maui. Las plazas para correr este triatlón se otorgan a los primeros clasificados en alguna de las otras pruebas del circuito Xterra.

El triatlón se convirtió en deporte olímpico en los Juegos Olímpicos de Sídney 2000 en la modalidad de corta distancia con drafting.

Dentro de la federación de triatlón se agrupan otros deportes que también se basan en la unión de varias disciplinas deportivas como:

- el duatlón (carrera a pie-ciclismo-carrera a pie),
- acuatlón (carrera a pie-natación-carrera a pie),
- triatlón de invierno (carrera a pie-ciclismo-esquí de fondo).

Dependiendo de la distancia del triatlón, tiene un nombre u otro:

- Distancia SuperSprint: 350m de natación + 10km de ciclismo + 2,5km de carrera a pie
- Distancia Sprint: 750m de natación + 20km de ciclismo + 5km de carrera a pie
- Distancia Estándar u Olímpica: 1,5km de natación + 40km de ciclismo + 10km de carrera a pie
- Media Distancia: 1,9-3km de natación + 80-90km de ciclismo + 20km de carrera a pie
- Larga Distancia: 1-4km de natación+ 100-200km de ciclismo+ 42km de carrera a pie (a este tipo pertenece el Ironman)



- Ultraman: 10km de natación + 415km de ciclismo+ 85km de carrera a pie (estos triatlones son los únicos en los que no se hacen todas las disciplinas seguidas. Se dividen en 3 días: el primer día se hace la natación y 145km de ciclismo, el segundo día se hacen los 270km de ciclismo restantes y el tercer día se corren 85km)

Variantes:

Existe el triatlón cross (natación, BTT y trail running), también conocido como triatlón de montaña o triatlón todoterreno, es una modalidad muy popular que combina natación en aguas abiertas, ciclismo de montaña (BTT) y trail running.

Se sustituye la bicicleta de carretera por la BTT y la carrera a pie por caminos de montaña, manteniendo el orden natación-ciclismo-carrera sin descanso. Las carreras de la marca Xterra son el referente mundial en esta disciplina. El formato estándar suele ser: Natación: 1 km (aproximadamente, en aguas abiertas), BTT: Suele ser la parte más exigente, con desnivel técnico, y Trail running: 6 km (en formato cross estándar).

La complejidad del triatlón radica en la necesidad de poder adquirir un dominio técnico y físico correcto en tres disciplinas diferentes, así como en sus grandes exigencias fisiológicas. Ellas son la base para poder llevar a cabo un posterior trabajo a nivel táctico que interrelacione las tres. (Fuente: <https://es.wikipedia.org/wiki/Triatl%C3%B3n>)

Categorías de triatlón

Se establece una **categoría absoluta** masculina y otra femenina:

Edad	Categoría
24-39	Absoluta

Además de la categoría absoluta a la que pertenecen todos los deportistas, estos podrán pertenecer a las siguientes categorías según sus edades e independientemente de su sexo:

Edad	Categoría
14-15	Cadetes
16-17	Juvenil
18-19	Júnior
20-23	Sub-23
40-49	Veterano 1
50-59	Veterano 2
60 o más años	Veterano 3



La categoría de edad estará determinada por la edad del participante el 31 de diciembre del año en que se celebre la competición. Como alternativa a las categorías relacionadas puede establecerse una competición por grupos de edad. En este caso los grupos de edad se agruparán de la siguiente manera: 1. xx-19 años 2. 20-24 años 3. 25-29 años 4. 30-34 años, etc.

Se podrán organizar campeonatos y competencias oficiales para las siguientes categorías y sobre las siguientes distancias:

- Cadete - Distancia Sprint
- Júnior - Distancias Sprint y Estándar

En una competición, todos los deportistas optan a la clasificación absoluta además de a la propia de su edad, si existiera hubiera. Ningún deportista podrá competir en una categoría diferente a la de su edad. Las Categorías en **Edad Escolar**:

Edad	Categoría
7	Pre-benjamín
8-9	Benjamín
10-11	Alevín
12-13	Infantil
14-15	Cadetes

Después de revisar el contexto de las competencias de Triatlón, se requiere diseñar e implementar un aplicativo basado en microservicios desarrollados en Spring Boot y Java Script.

En primera instancia, y considerando que un triatleta se registra, cambia o actualiza sus datos de forma total o parcial, o en último caso, se da de baja como deportista de estas justas; el aplicativo debe proveer los respectivos microservicios que permitan realizar estos procesos. Tenga en cuenta que:

- a) Creación del perfil del triatleta: El deportista crea un perfil que incluye su nombre, identificación, edad, género, categoría según edad.
- b) Configuración de preferencias del triatleta: Se establecen las preferencias de participación tales como: especialidad de triatlón según distancia, y si participa en modalidad cross o no.

Para seleccionar las preferencias, el FrontEnd debe presentar dichas preferencias en un carrito de imágenes, permitiendo deslizamiento (Swiping), dando clic y almacenando la especialidad.

Seguidamente, el aplicativo también deberá proveer los siguientes microservicios sobre el triatleta:

- 1) Modificar una característica particular del triatleta (nombre, identificación, o categoría). Deben proveerse los tres microservicios.



- 2) Consultar el triatleta por identificación
- 3) Consultar grupos de triatletas por género, categoría, especialidad, modalidad cross. Deben proveerse los cuatro microservicios.
- 4) Eliminar un triatleta

En la visualización desplegada en el FrontEnd para los microservicios del triatleta, debe aparecer la información del usuario, junto con todas sus características. Permitiendo adicionalmente, usar los diferentes microservicios tanto para el usuario (insertar, modificar, eliminar, etc., y demás servicios CRUD y consultas especificadas que se realicen bajo Spring Boot). A continuación se explicitan algunos requerimientos:

- Para el manejo del controlador, puede usar `@RestController` o `@Controller`. Sin embargo, tenga en cuenta que la información de request y response de los microservicios debe ser desplegada en el FrontEnd.
- Las páginas a usar no deben estar almacenadas en `main/resources/templates`. Por el contrario, deben manejarse en un proyecto Html externo. Tanto el backend como el FrontEnd deben ser desarrollados en Netbeans.
- Para el almacenamiento de los datos de los triatletas, se debe hacer uso de MySQL como base de datos (con el fin de no estar recargándola constantemente), la cual deberá ser mapeada usando JPA.
- La comunicación desde el FrontEnd hacia el Backend y viceversa, deberá hacerse usando JavaScript, específicamente usando peticiones asíncronas tipo Fetch a cada uno de los endpoint expuestos en el backend. Los mapeos de los endpoint deben realizarse haciendo uso de la anotación `@RequestMapping` como mapeo base de la clase y como método de mapeo de cada método.
- En el backend se debe conservar la respectiva separación de responsabilidades entre: controlador, repositorio, servicios y modelo.
- Se deben conservar y manejar las políticas CORS para la comunicación entre el front (proyecto html) y el back (proyecto Spring Boot)
- El diseño usado en el FrontEnd deberá aportar una imagen institucional y contenido acerca del deporte, los participantes, los tipos de categorías, etc. Puede usar hojas de estilo, plantillas descargables en la web o frameworks como Bootstrap.
- Los elementos del FrontEnd deben ser decorados haciendo uso de CSS y efectos JavaScript. No se avalan diseños simplistas ni minimalistas. Debe presentarse la información del deporte y de los atletas en todo su esplendor.

Es importante tener en cuenta que:

- Tenga en cuenta que su diseño debe ser escalable, lo que implica que a futuro, de la jerarquía de herencia de Persona – Atleta, también se podría desprender Persona – Juez, por ende, su diseño y anotaciones deberá manejar la herencia respectiva.
- El registro de un atleta debe manejar la foto del mismo.



- Al registrarse un triatleta se le debe enviar un correo de registro. Para tal fin puede usar JavaMailServer de Spring (<https://docs.spring.io/spring-boot/reference/io/email.html>).
- La visualización del FrontEnd deberá actualizarse ante cualquier evento realizado por los microservicios.

Considere adicionalmente que debe utilizar las anotaciones y/o parametrizaciones respectivas de

- Lombok y thymeleaf
- Validation: dependencia que permite validar ciertos parámetros del usuario o de procesos. Fuente: <https://dev.to/gianfcop98/spring-boot-and-validation-a-complete-guide-with-valid-and-validated-471p>
- Debe manejar las políticas CORS, dado que el proyecto FrontEnd estará en un proyecto HTML, mientras que el BackEnd se encontrará en un proyecto Spring Boot.
- Para la construcción del FrontEnd, puede usar plantillas de html descargadas de repositorios, y ajustadas debidamente. Por otro lado, use hojas de estilos, JSP, y Java Script para realizar la decoración y procesamiento en el FrontEnd.

En cuanto al BackEnd y FrontEnd, se deben proveer:

- Los controles (con sus respectivos endpoints y mapeos), modelos y repositorios respectivos.
- El envío de correo se debe desarrollar como investigación para el parcial.
- Debe tener la separación SOLID respectiva (Controladores, Servicios, Repositorios y modelos)
- No se recibirán páginas planeas, llenas de texto, con enlaces href. Para tal fin puede usar los iconos de las fuentes de Google (<https://fonts.google.com/icons>), y a estos iconos asignarles las respectivas acciones.
- El diseño de las páginas debe ser visualmente agradable, decorado, estéticamente presentable, y tener una excelente presentación gráfica y visual. Además, debe ofrecer toda la interactividad posible a través de menús, botones, tablas creadas dinámicamente, etc. Puede usar plantillas o templates publicados en la Web, dando el respectivo crédito.
- Toda la información debe ser diligenciada manualmente al momento de la ejecución del proyecto.
- El proyecto debe contar con un esquema de navegación que permita acceder a todas las páginas, y de igual forma, regresar siempre a la página principal.
- Cualquier plugin, librería, etc., necesaria para el funcionamiento del proyecto, debe ser cargada en el mismo. Si al momento de cargar el proyecto, este no funciona por falta de alguna librería, por ejemplo, NO se realizará ninguna configuración para intentar ver si el proyecto funciona. Será catalogado como entrega que no funciona, y por ende, tendrá una nota de 0.0
- En la sustentación se evaluará adicionalmente el uso e interacción de la base de datos seleccionada, por ejemplo, cambiando a través de SQL algún parámetro de una publicación o usuario, y su correspondiente visualización actualizada a través de los endpoints.

Referencias de Soporte

- Templates HTML: <https://formbold.com/templates>
- Templates HTML: <https://codepen.io/>



- Guía de Procesos Spring: <https://spring.io/guides>
- Conexión entre FrontEnd y Backend con Spring Boot y una API REST para realizar un CRUD: https://www.youtube.com/watch?v=00_36HlaicA&ab_channel=AraOverFlow
- Ejemplo de consumo Spring: <https://sacavix.com/2020/03/consumo-de-api-rest-con-spring-boot/>
- Manejo de fetch: <https://www.freecodecamp.org/espanol/news/fetch-api-como-realizar-un-get-request-y-un-post-request-en-javascript/>
- Conexión entre Frontend y Backend con Spring Boot y una API REST: https://www.youtube.com/watch?v=00_36HlaicA&ab_channel=AraOverFlow
- Problemas con CORS: https://www.youtube.com/watch?v=jJTfuiyEqb8&ab_channel=YoAndroide
- Cómo realizar un GET Request y un POST Request en JavaScript: <https://www.freecodecamp.org/espanol/news/fetch-api-como-realizar-un-get-request-y-un-post-request-en-javascript/>
- Spring desde 0: <https://jose-antonio-henriquez-chavarria.gitbook.io/spring-desde-0>
- Desarrollo web con Spring - Consumo de api de Spring con Javascript: https://www.youtube.com/watch?v=goejcibleGs&ab_channel=JandroCode

Aspectos a tener en cuenta

- a) El aplicativo debe estar organizado en paquetes como se explicó en la arquitectura MVC. Si no cumple con este requerimiento, a la nota definitiva del corte se le descontará una unidad completa.
- b) El aplicativo debe cumplir con los principios de diseño de clases (específicamente los SOLID). Si no cumple con este requerimiento, a la nota definitiva del corte se le descontará una unidad completa.
- c) El aplicativo debe contar con buenas prácticas de documentación. Aplique las anotaciones respectivas ofrecidas por JavaDoc. TODOS los elementos del aplicativo deben estar documentados. Si no cumple con este requerimiento, a la nota definitiva del corte se le descontará una unidad completa.
- d) El aplicativo debe contar con la aplicación de estándares de codificación. Si no cumple con este requerimiento, a la nota definitiva del corte se le descontará una unidad completa.
- e) La entrega debe llevar una imagen JPG del diagrama de Clases del proyecto, almacenada en la carpeta "Specs". Este diagrama de clases debe estar organizado en paquetes. Lo especificado en el diagrama de clases DEBE corresponder a lo desarrollado en el aplicativo. De lo contrario se le descontará una unidad a la nota definitiva.
- f) La entrega constará de dos partes: el producto de software funcionando, la sustentación del mismo. Si el estudiante hace la entrega, pero no identifica sus componentes, no encuentra los requerimientos o sus partes, su entrega se anulará, y su nota será 0.0.
- g) La entrega la subirá una sola persona del grupo, y adicionará un txt con los nombres de los integrantes del grupo (en un txt ubicado en la carpeta "Docs" dentro del proyecto). Si no se adiciona este documento en dicha ubicación, esta entrega será registrada únicamente bajo el nombre de la persona que subió el proyecto al aula. Las otras personas tendrán nota de 0.0 en el 100% de la nota del corte, y será catalogado como no entregado. Las entregas remitidas por correo electrónico, serán borradas y no se tendrán en cuenta, y su nota de entrega será 0.0. No se aceptará que los nombres de los integrantes vengan en los comentarios del aula, o en un correo, o dentro de alguna clase o imagen.
- h) Evítese plagio entre compañeros.



- i) El examen se realizará en los grupos definidos en las reglas del curso. A pesar de que el parcial es en grupo, la sustentación será individual, y se computarán las notas de sustentación individual para obtener una sola, que se sumará a la entrega: Por esta razón, es necesario que las personas del grupo tengan conocimiento del proyecto y sustenten las respectivas preguntas.
- j) La sustentación puede ser: completar un segmento de código borrado, argumentar como se desarrolló un requerimiento, explicar un conjunto de líneas de código, etc.
- k) En la web puede encontrar clases, métodos o librerías que le permitan resolver algún requerimiento, pero al implementarlas deben tener claro conocimiento de su uso y aplicación, para poder sustentar la entrega. Al igual, se debe dar crédito respectivo al autor de dicho código.
- l) Si requiere de alguna librería adicional, o de algún plugin, recuerde traerlo e instalarlo para la sustentación, además de adjuntarlo a la entrega por el aula.
- m) Entrega que no esté registrada en el aula, no será recibida. En la sustentación es lo primero que se revisará. Si no tiene usuario activo en UDIN, por favor realizar la gestión pertinente para activarlo. Cualquier correo con la entrega del proyecto, recibido anterior o posterior a la entrega, será eliminado y su nota será de 0.0. La entrega debe contener el proyecto comprimido con todas las clases necesarias. No se recibirá como envío un enlace a un drive. Si sube al aula cualquier otra cosa que no sea el proyecto completo y comprimido, su nota será de 0.0.
- n) Los turnos de sustentación serán sorteados la clase previa a la evaluación.
- o) Toda la comunicación del BackEnd debe ser expuesta por el FrontEnd, bien sea a través de páginas html o de envíos JSON. No se admiten envíos de mensajes con ventanas emergentes o con salidas en consola. De lo contrario se le descontará una unidad a la nota definitiva.
- p) El FrontEnd debe estar construido y usado en un proyecto Html externo al proyecto Spring. De lo contrario se le descontará una unidad a la nota definitiva.
- q) Tengan en cuenta la hora y fecha de la entrega en el aula. Suba su entrega por lo menos unos 30 minutos antes de la fecha y hora fijadas. Si no subió su entrega al aula en la hora y fecha indicada, así argumente que trato de subirla un minuto antes pero que la velocidad de internet lo dejó por fuera, etc., su entrega total (Sustentación y Entrega), será evaluada como “No entregada” con nota total de 0.0.
- r) El FrontEnd debe tener un adecuado uso de elementos Html, Hojas de Estilo, etc.
- s) Por último, y dado el uso que se le puede dar a la Inteligencia Artificial (IA), cualquier indicio del uso de IA, por más mínimo que este sea, generará que el proyecto sea invalidado por fraude. De acuerdo al estatuto estudiantil, un fraude en una prueba debe ser reportado, la prueba anulada en su totalidad, y es causal de matrícula condicional.

Profesor: Jhon Herrera Cubides